

НИУ ИТМО
Факультет ПИиКТ

Лабораторная работа №5

Интерполяция функции

Вариант №5

Преподаватель **Наумова Надежда Александровна**
Подготовил **Лоскутов Прохор Александрович**

Санкт - Петербург 2026

Оглавление

Оглавление	2
1 Цель работы	3
2 Порядок выполнения работы	3
3 Рабочие формулы методов	3
3.1 Постановка задачи	3
3.2 Многочлен Лагранжа	4
3.3 Конечные разности	4
3.4 Интерполяционные формулы Ньютона	4
3.5 Интерполяционные формулы Гаусса	4
3.6 Формула Стирлинга	5
3.7 Формула Бесселя	5
3.8 Оценка погрешности	5
4 Вычислительная реализация задачи	6
4.1 Таблица конечных разностей	6
4.2 Первая формула Ньютона в точке $X_1 = 2.112$	6
4.3 Первая формула Гаусса в точке $X_2 = 2.205$	7
4.4 Многочлен Лагранжа в точках X_1 и X_2	8
4.5 Сравнение результатов	8
5 Программная реализация задачи	9
5.1 Общая схема расчёта	9
5.2 Блок-схемы методов	12
5.3 Архитектура модуля	16
5.4 Исходный код основных элементов расчёта	18
5.5 Интерфейс приложения	25
6 Выводы	25

1 Цель работы

Решить задачу интерполяции: по таблично заданной функции $y = f(x)$ построить интерполяционный многочлен и найти приближённые значения функции в точках X_1 и X_2 , отличных от узловых. Для исследования использовать многочлены **Лагранжа**, **Ньютона** (с конечными разностями) и **Гаусса**; дополнительно — схемы **Стирлинга** и **Бесселя**.

2 Порядок выполнения работы

1. Выбрать по варианту таблицу $y = f(x)$ (вариант №5 — таблица 1.5, семь равноотстоящих узлов с шагом $h = 0.05$).
2. Построить таблицу конечных разностей.
3. Вычислить значение функции для аргумента $X_1 = 2.112$ по первой интерполяционной формуле Ньютона.
4. Вычислить значение функции для аргумента $X_2 = 2.205$ по первой интерполяционной формуле Гаусса.
5. Вычислить значения функции в точках X_1 и X_2 с помощью многочлена Лагранжа, сравнить результаты.
6. Реализовать программу: ввод данных тремя способами (таблица с клавиатуры, файл, встроенная функция), построение таблицы разностей, расчёт всеми методами, построение графика.

3 Рабочие формулы методов

3.1 Постановка задачи

Функция задана таблицей значений (x_i, y_i) , $i = 0, 1, \dots, n$. Требуется построить многочлен $P_n(x)$ степени не выше n , удовлетворяющий условию интерполяции $P_n(x_i) = y_i$, и вычислить его значение в промежуточной точке. Такой многочлен единственен, поэтому все рассмотренные ниже методы при полном наборе узлов дают один и тот же результат — различаются лишь формой записи и удобством вычислений.

3.2 Многочлен Лагранжа

$$L_n(x) = \sum_{i=0}^n y_i \prod_{\substack{j=0 \\ j \neq i}}^n \frac{x - x_j}{x_i - x_j}$$

Применим для произвольных (в том числе неравноотстоящих) узлов.

3.3 Конечные разности

Для равноотстоящих узлов ($x_i = x_0 + ih$) вводятся конечные разности:

$$\Delta^0 y_i = y_i, \quad \Delta^k y_i = \Delta^{k-1} y_{i+1} - \Delta^{k-1} y_i$$

3.4 Интерполяционные формулы Ньютона

Обозначим $t = (x - x_0)/h$. **Первая формула** (интерполирование вперёд, для левой части таблицы):

$$N_n(x) = y_0 + t\Delta y_0 + \frac{t(t-1)}{2!} \Delta^2 y_0 + \dots + \frac{t(t-1)\dots(t-n+1)}{n!} \Delta^n y_0$$

При $t = (x - x_n)/h$ — **вторая формула** (интерполирование назад, для правой части таблицы):

$$N_n(x) = y_n + t\Delta y_{n-1} + \frac{t(t+1)}{2!} \Delta^2 y_{n-2} + \dots + \frac{t(t+1)\dots(t+n-1)}{n!} \Delta^n y_0$$

3.5 Интерполяционные формулы Гаусса

Узел $x_0 = a$ выбирается ближайшим к x ; $t = (x - x_0)/h$. **Первая формула** ($x > a$):

$$P_n(x) = y_0 + t\Delta y_0 + \frac{t(t-1)}{2!} \Delta^2 y_{-1} + \frac{(t+1)t(t-1)}{3!} \Delta^3 y_{-1} + \frac{(t+1)t(t-1)(t-2)}{4!} \Delta^4 y_{-2} + \dots$$

Вторая формула ($x < a$):

$$P_n(x) = y_0 + t\Delta y_{-1} + \frac{t(t+1)}{2!}\Delta^2 y_{-1} + \frac{(t+1)t(t-1)}{3!}\Delta^3 y_{-2} + \frac{(t+2)(t+1)t(t-1)}{4!}\Delta^4 y_{-2} + \dots$$

3.6 Формула Стирлинга

Среднее арифметическое первой и второй формул Гаусса (нечётное число узлов, применяется при $|t| \leq 0.25$):

$$P_n(x) = y_0 + t\frac{\Delta y_{-1} + \Delta y_0}{2} + \frac{t^2}{2!}\Delta^2 y_{-1} + \frac{t(t^2-1)}{3!}\frac{\Delta^3 y_{-2} + \Delta^3 y_{-1}}{2} + \frac{t^2(t^2-1)}{4!}\Delta^4 y_{-2} + \dots$$

3.7 Формула Бесселя

Интерполирование «на середину» (чётное число узлов, применяется при $0.25 \leq t \leq 0.75$):

$$P_n(x) = \frac{y_0 + y_1}{2} + \left(t - \frac{1}{2}\right)\Delta y_0 + \frac{t(t-1)}{2!}\frac{\Delta^2 y_{-1} + \Delta^2 y_0}{2} + \frac{\left(t - \frac{1}{2}\right)t(t-1)}{3!}\Delta^3 y_{-1} + \dots$$

3.8 Оценка погрешности

Остаточный член интерполяции для любого из методов:

$$R_n(x) = f(x) - P_n(x), \quad |R_n(x)| \leq \frac{M_{n+1}}{(n+1)!} \left| \prod_{i=0}^n (x - x_i) \right|$$

$$M_{n+1} = \max_{x \in [x_0, x_n]} |f^{(n+1)}(x)|$$

Поскольку о функции известна только таблица, на практике степень многочлена принимают равной порядку практически постоянных конечных разностей.

4 Вычислительная реализация задачи

По варианту №5 выбрана таблица 1.5 — семь равноотстоящих узлов с шагом $h = 0.05$; точки интерполяции $X_1 = 2.112$ и $X_2 = 2.205$.

Таблица 1 — заданная табличная функция (вариант 5)

i	0	1	2	3	4	5	6
x_i	2.10	2.15	2.20	2.25	2.30	2.35	2.40
y_i	3.7587	4.1861	4.9218	5.3487	5.9275	6.4193	7.0839

4.1 Таблица конечных разностей

Таблица 2 — конечные разности $\Delta^k y_i$

i	x_i	y_i	Δy_i	$\Delta^2 y_i$	$\Delta^3 y_i$	$\Delta^4 y_i$	$\Delta^5 y_i$	$\Delta^6 y_i$
0	2.10	3.7587	0.4274	0.3083	-0.6171	1.0778	-1.7774	2.9757
1	2.15	4.1861	0.7357	-0.3088	0.4607	-0.6996	1.1983	
2	2.20	4.9218	0.4269	0.1519	-0.2389	0.4987		
3	2.25	5.3487	0.5788	-0.0870	0.2598			
4	2.30	5.9275	0.4918	0.1728				
5	2.35	6.4193	0.6646					
6	2.40	7.0839						

Конечные разности не стабилизируются (модуль $\Delta^k y_0$ растёт от 0.43 до 2.98), то есть таблица не порождена гладкой функцией невысокой степени; это заранее указывает на колебательный характер интерполяционного многочлена высокой степени.

4.2 Первая формула Ньютона в точке $X_1 = 2.112$

Точка $X_1 = 2.112$ лежит у левого края таблицы, поэтому применяем первую формулу (интерполирование вперёд) с опорным узлом $x_0 = 2.10$:

$$t = \frac{X_1 - x_0}{h} = \frac{2.112 - 2.10}{0.05} = 0.24$$

Каждое слагаемое равно $c_k \cdot \Delta^k y_0$, где $c_k = (t(t-1)\cdots(t-k+1))/k!$ — множитель при k -й конечной разности (берётся первая строка таблицы 2):

Таблица 3 — вычисление по первой формуле Ньютона ($X_1 = 2.112$,
 $t = 0.24$)

k	c_k	$\Delta^k y_0$	$c_k \cdot \Delta^k y_0$
0	1.000000	3.7587	+3.758700
1	0.240000	0.4274	+0.102576
2	−0.091200	0.3083	−0.028117
3	0.053504	−0.6171	−0.033017
4	−0.036918	1.0778	−0.039790
5	0.027762	−1.7774	−0.049344
6	−0.022025	2.9757	−0.065539
Σ			3.645469

Итого $N_6(2.112) = \mathbf{3.645469}$.

4.3 Первая формула Гаусса в точке $X_2 = 2.205$

Точка $X_2 = 2.205$ расположена в центральной части таблицы. Ближайший узел — $x_2 = 2.20$, принимаем его за центральный ($a = x_0 = 2.20, i = 2$):

$$t = \frac{X_2 - a}{h} = \frac{2.205 - 2.20}{0.05} = 0.1 > 0$$

Так как $t > 0$, используем первую формулу Гаусса. В каждом слагаемом множитель c_k — это центральная t -комбинация, делённая на $k!$ (для 6-й разности слева от $a = x_2$ узла нет, поэтому ряд обрывается на 5-й разности):

Таблица 4 — вычисление по первой формуле Гаусса ($X_2 = 2.205$,
 $t = 0.1$)

k	c_k	Разность	Слагаемое
0	1.000000	$y_0 = 4.9218$	+4.921800
1	0.100000	$\Delta y_0 = 0.4269$	+0.042690
2	−0.045000	$\Delta^2 y_{-1} = -0.3088$	+0.013896
3	−0.016500	$\Delta^3 y_{-1} = 0.4607$	−0.007602
4	0.007838	$\Delta^4 y_{-2} = 1.0778$	+0.008447
5	0.003292	$\Delta^5 y_{-2} = -1.7774$	−0.005851
Σ			4.973381

Итого $P_5(2.205) = 4.973381$.

4.4 Многочлен Лагранжа в точках X_1 и X_2

По формуле Лагранжа $L_6(x) = \sum_{i=0}^6 y_i l_i(x)$, где $l_i(x) = \prod_{j \neq i} (x - x_j) / (x_i - x_j)$. Значения базисных многочленов и слагаемых:

Таблица 5 — слагаемые многочлена Лагранжа

i	$l_i(X_1)$	$y_i l_i(X_1)$	$l_i(X_2)$	$y_i l_i(X_2)$
0	0.528591	+1.986817	0.002955	+0.011106
1	1.001542	+4.192554	-0.033845	-0.141679
2	-1.081210	-5.321498	0.930742	+4.580928
3	0.919289	+4.917004	0.137888	+0.737520
4	-0.506098	-2.999897	-0.048986	-0.290367
5	0.159910	+1.026510	0.012838	+0.082410
6	-0.022025	-0.156020	-0.001591	-0.011271
Σ		3.645469		4.968647

4.5 Сравнение результатов

Таблица 6 — сводка вычислительной части

Точка	Метод	t	Значение
$X_1 = 2.112$	Ньютон, 1-я формула	0.24	3.645469
$X_1 = 2.112$	Лагранж	—	3.645469
$X_2 = 2.205$	Гаусс, 1-я формула	0.10	4.973381
$X_2 = 2.205$	Лагранж	—	4.968647

В точке X_1 значения по Ньютону и Лагранжу совпадают полностью (3.645469): оба используют все семь узлов и строят один и тот же многочлен 6-й степени. В точке X_2 Лагранж (полная 6-я степень) даёт 4.968647, а первая формула Гаусса от центрального узла $a = 2.20$ обрывается на 5-й разности (для 6-й нет узла слева) и даёт 4.973381; расхождение ≈ 0.005 объясняется именно разной степенью усечения и подтверждает корректность обоих расчётов.

5 Программная реализация задачи

Модуль «Интерполяция» добавлен в общее приложение (Qt6, QML + C++). Исходные данные задаются **тремя способами**:

Способ ввода	Описание
Таблица с клавиатуры	Редактируемая таблица (x_i, y_i) , число узлов задаётся кнопками $\pm$
Файл данных	Три встроенных набора (вариант 5, $\sin x$, функция Рунге) и диалог открытия произвольного файла с парами « x y »
Встроенная функция	Выбор функции ($\sin x$, $\cos x$, e^x , $1/(1+x^2)$, $x \sin x$), интервала $[a, b]$ и числа узлов; таблица генерируется программой

Программа строит таблицу конечных разностей, вычисляет значение в точке X^* всеми методами (Лагранж, Ньютон вперёд/назад, Гаусс I/II, Стирлинг, Бессель) и сравнивает их. Для конечно-разностных методов проверяется равномерность сетки: если узлы неравноотстоящие, считается только многочлен Лагранжа, а для остальных методов в таблице выводится пометка «Неприменимо: нужны равноотстоящие узлы». График показывает узлы интерполяции, интерполяционный многочлен и (в режиме функции) заданную функцию разными цветами.

5.1 Общая схема расчёта

Процедура `run` проверяет данные, для Лагранжа считает сразу, а для остальных методов — после проверки равномерности сетки и построения таблицы разностей — вызывает соответствующую подпрограмму.

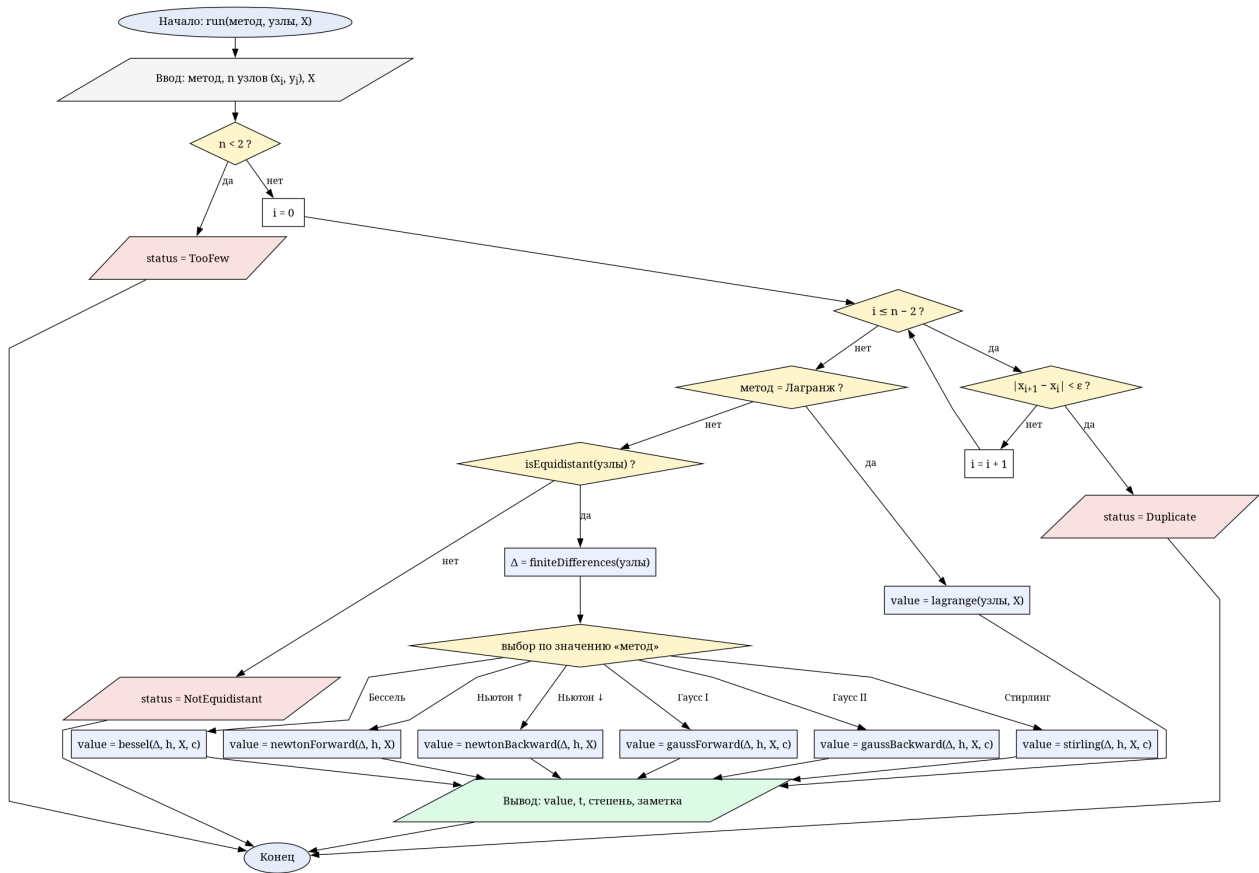


Рис. 1. Общая блок-схема расчёта run

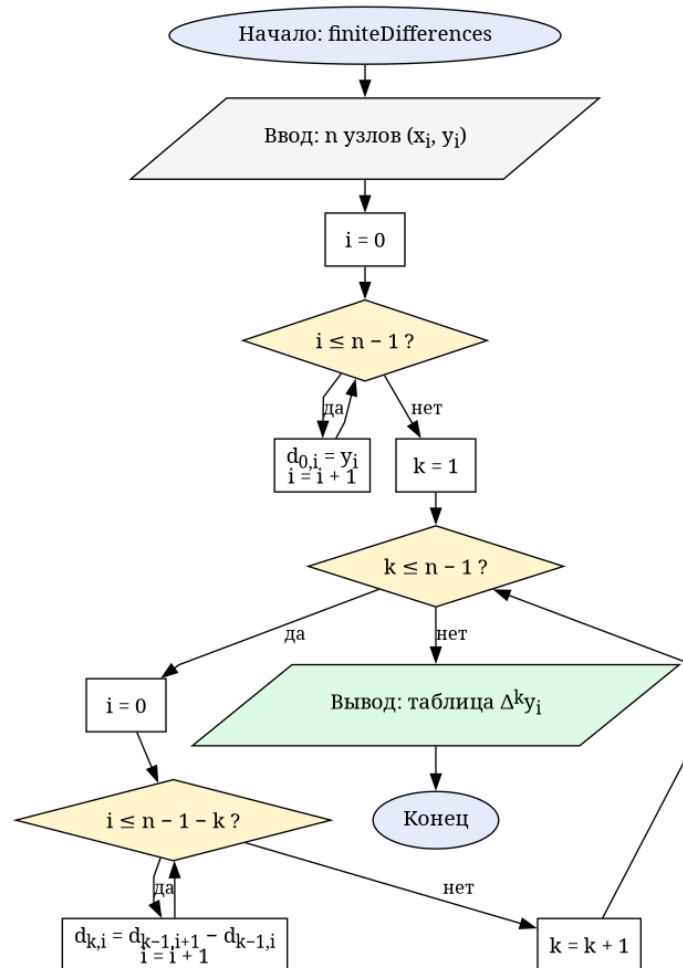


Рис. 2. Построение таблицы конечных разностей `finiteDifferences`

5.2 Блок-схемы методов

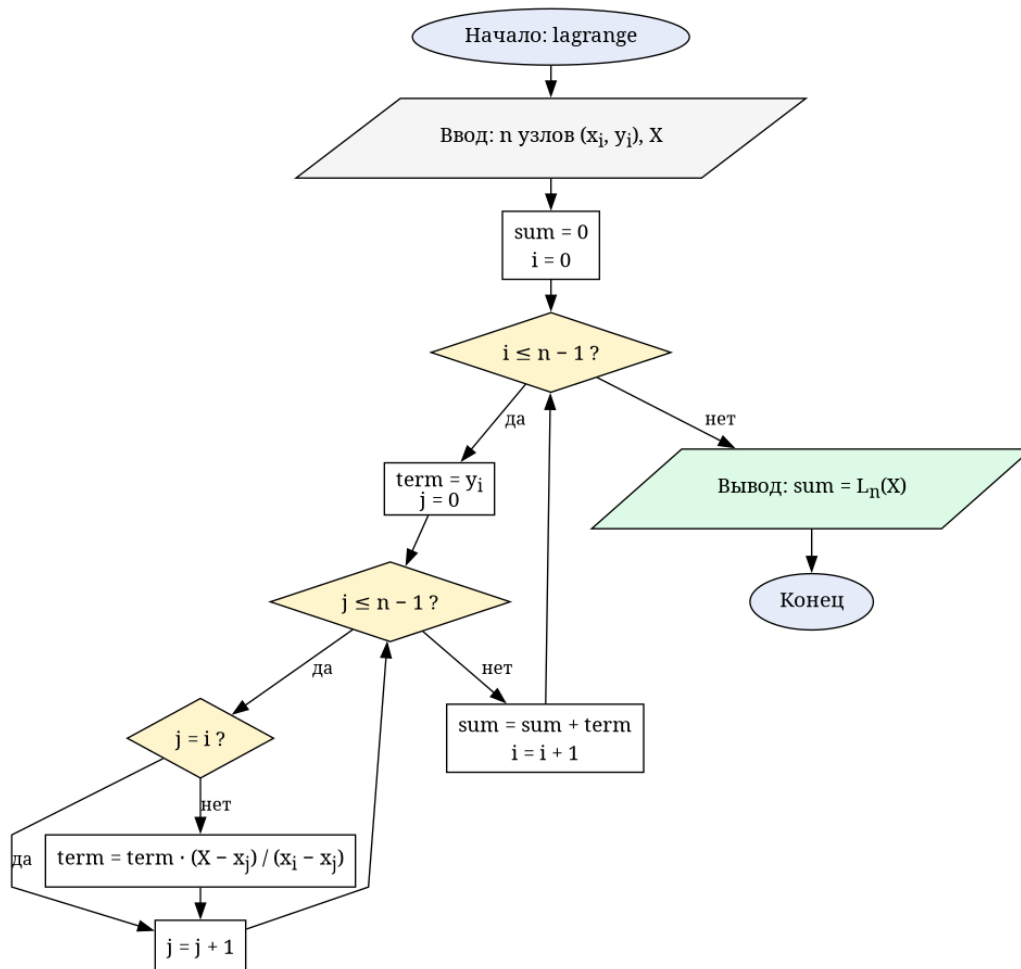


Рис. 3. Многочлен Лагранжа lagrange

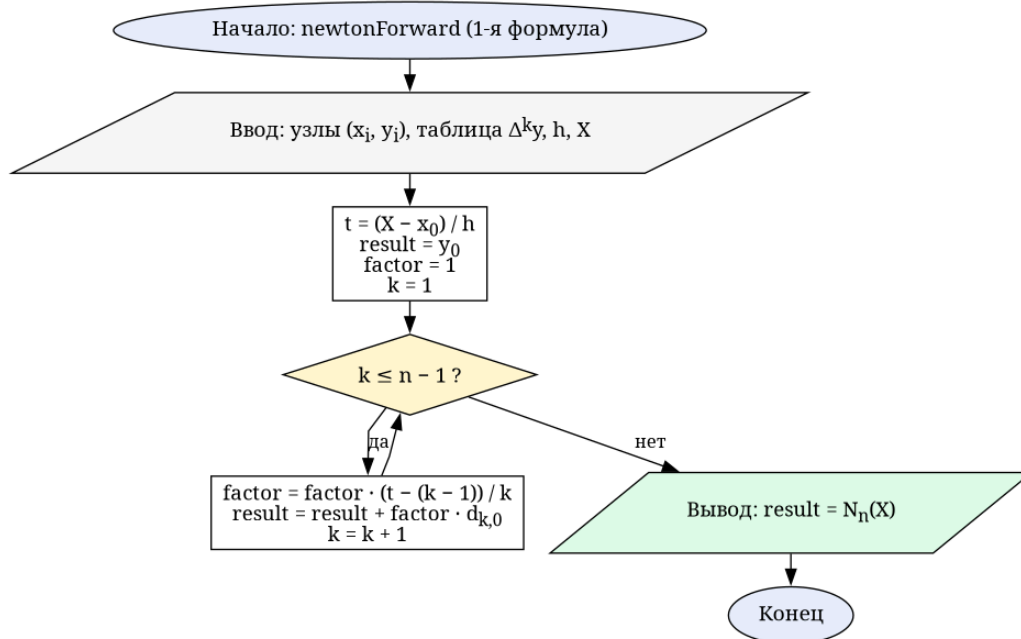


Рис. 4. Первая формула Ньютона newtonForward

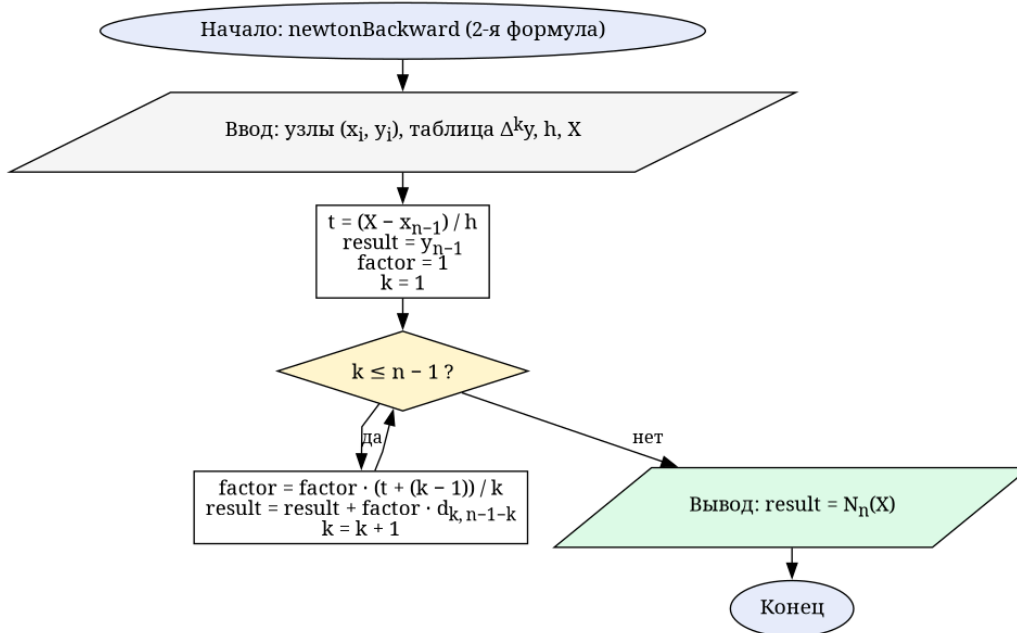


Рис. 5. Вторая формула Ньютона newtonBackward

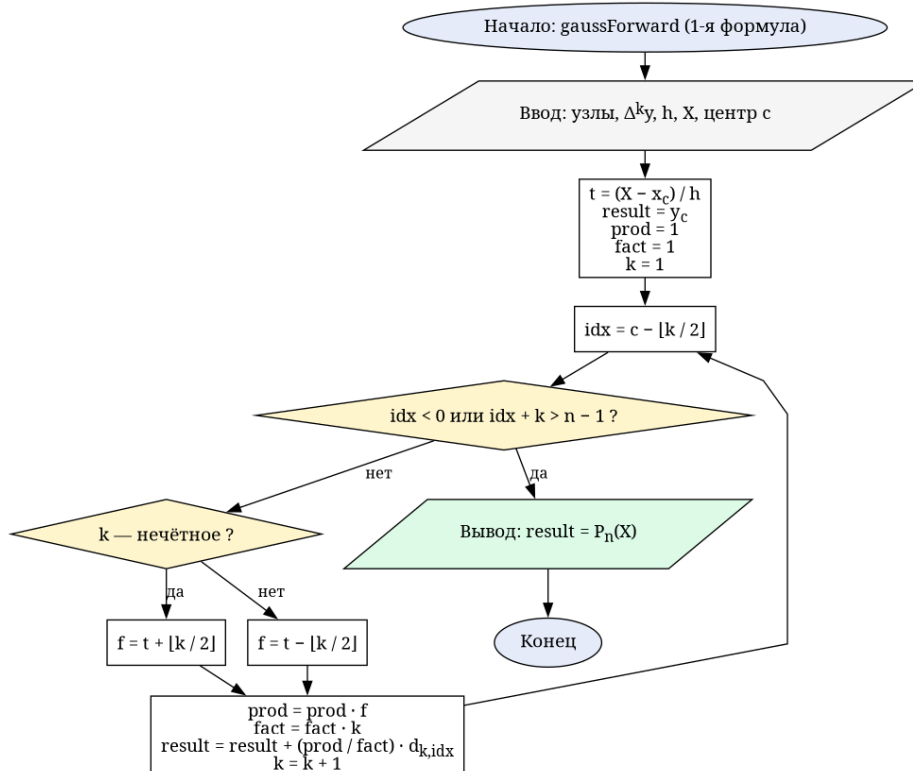


Рис. 6. Первая формула Гаусса gaussForward

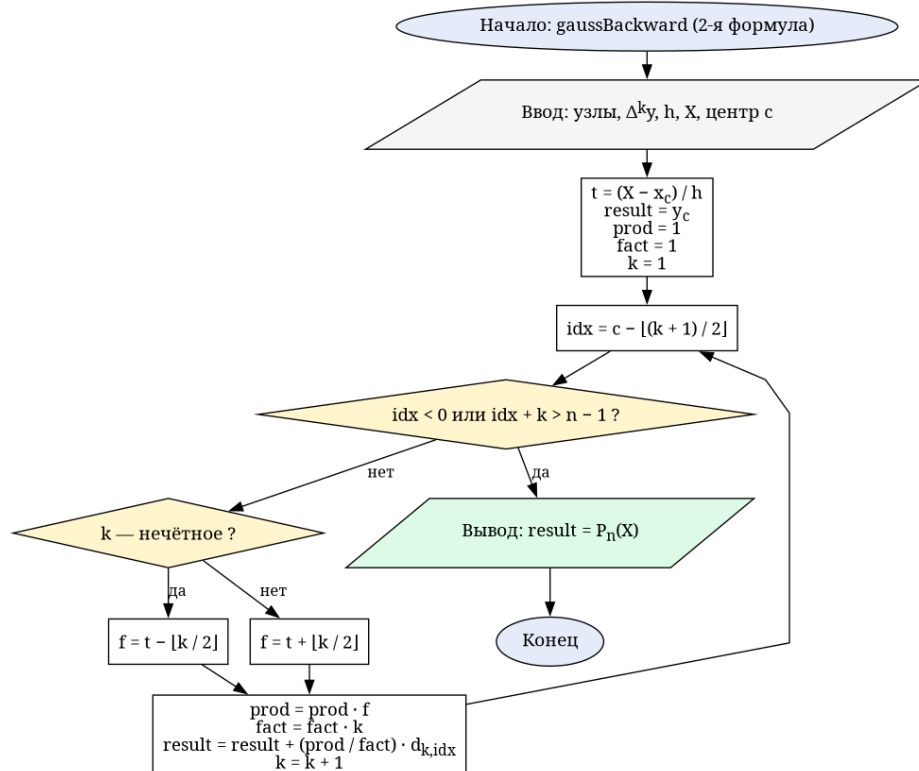


Рис. 7. Вторая формула Гаусса gaussBackward

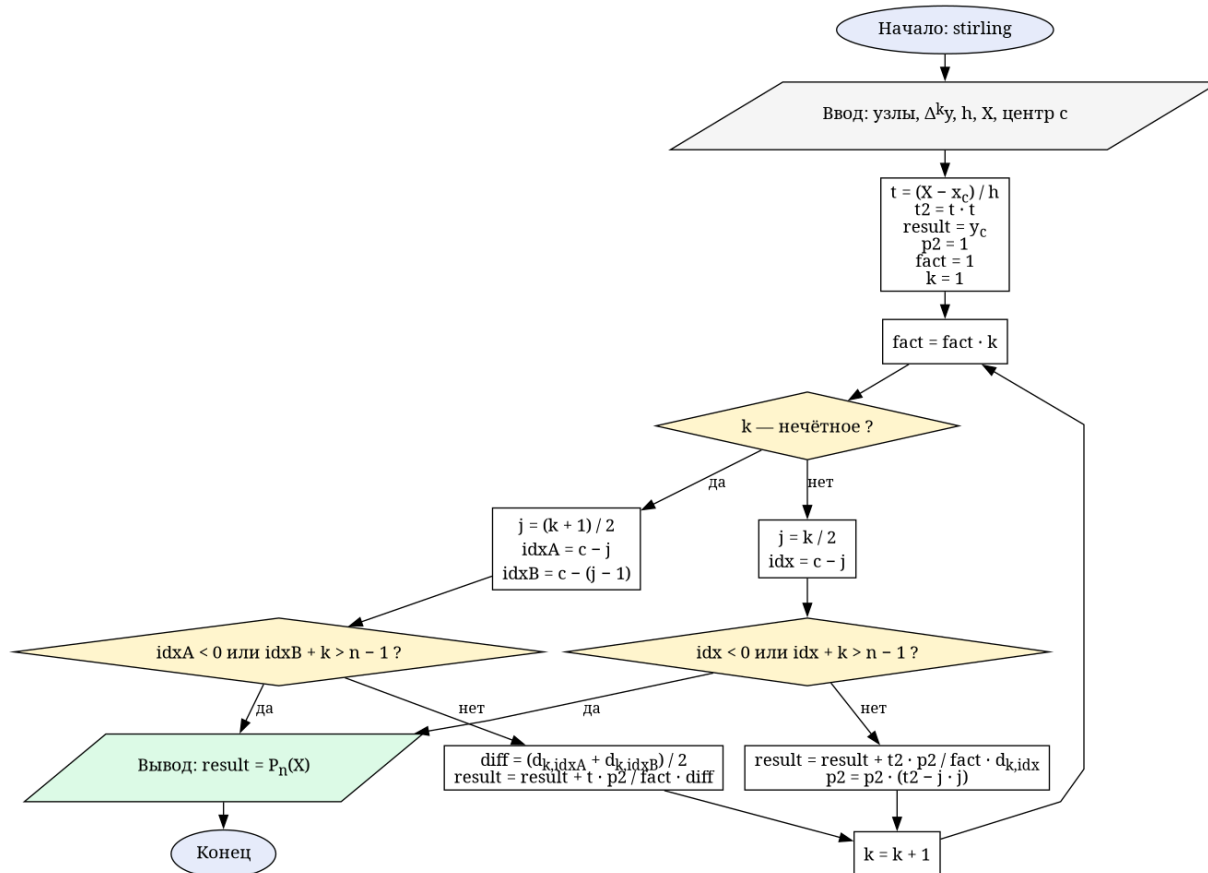


Рис. 8. Формула Стирлинга stirling

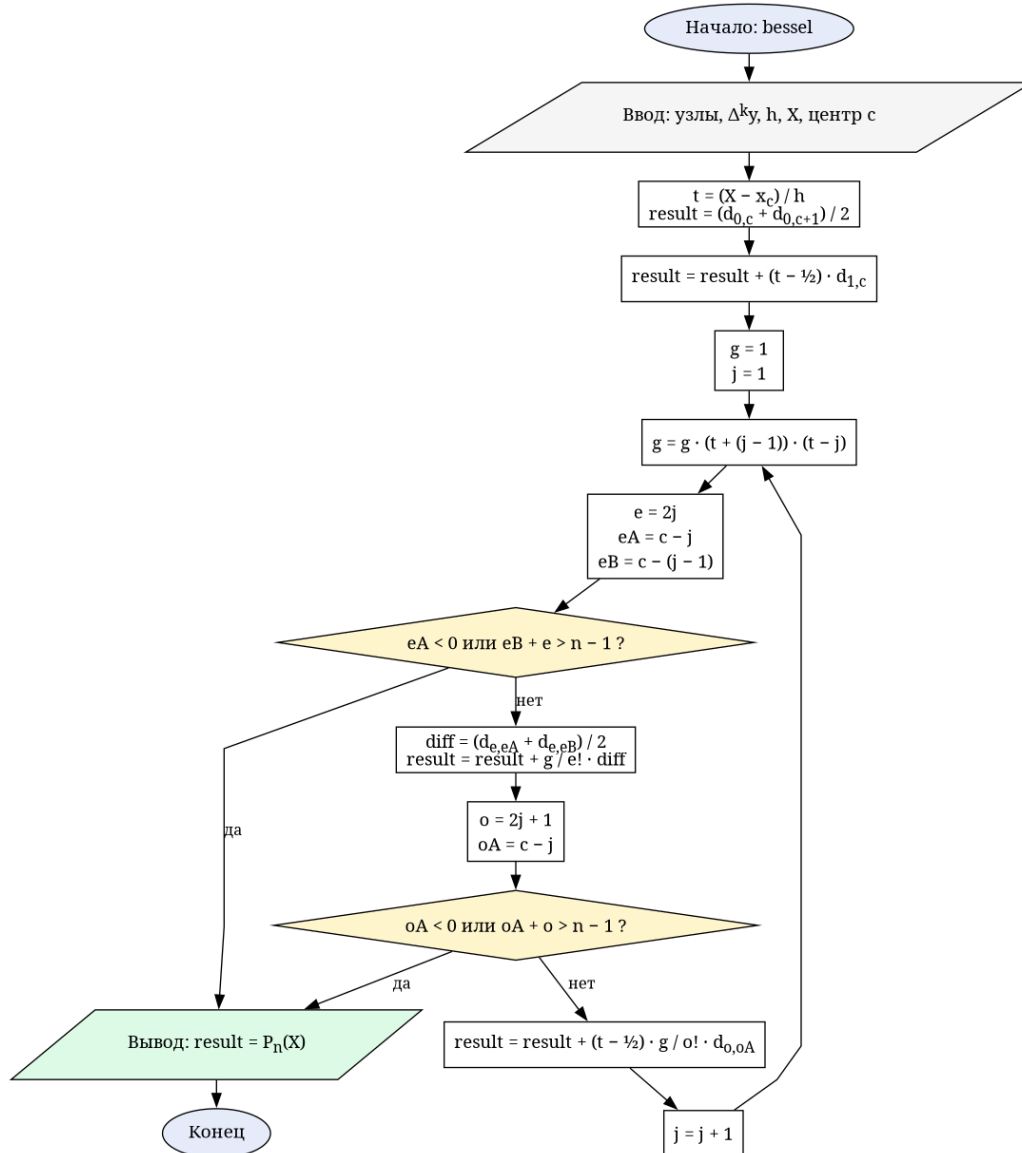


Рис. 9. Формула Бесселя `bessel`

5.3 Архитектура модуля

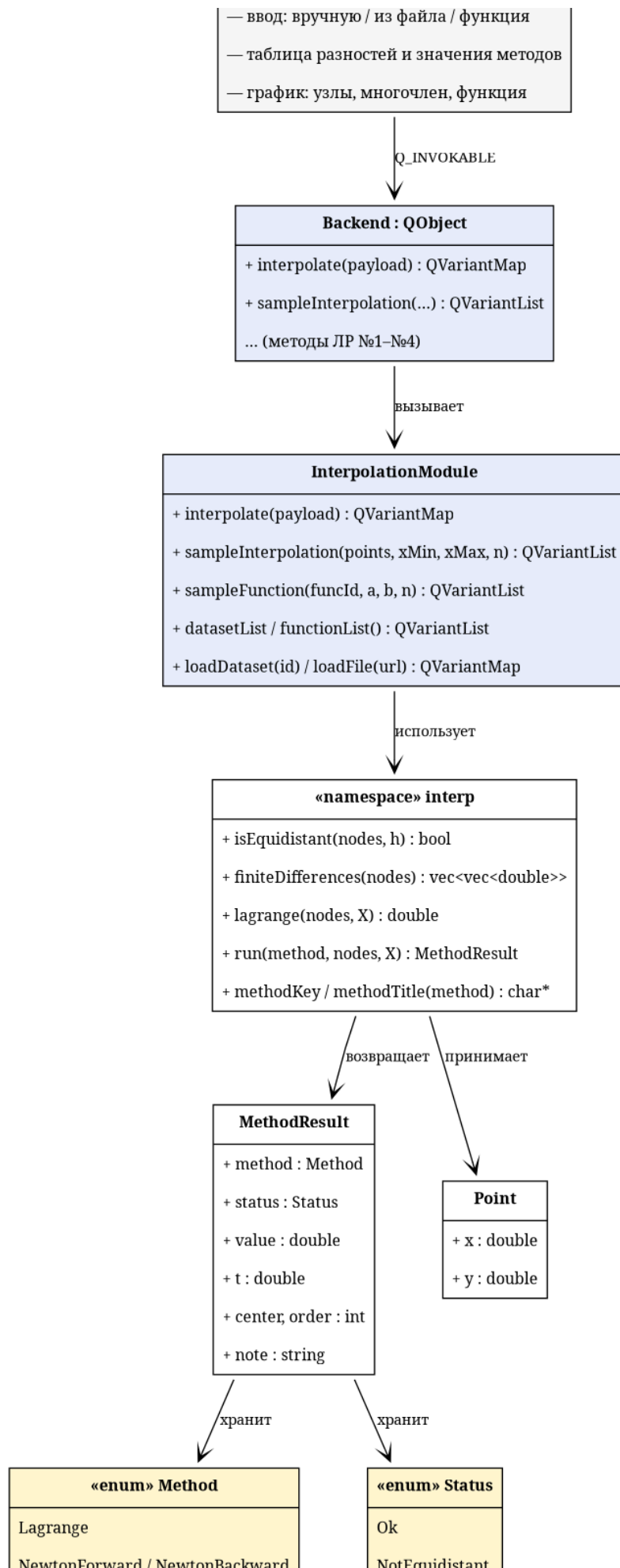


Рис. 10. Диаграмма классов модуля интерполяции

5.4 Исходный код основных элементов расчёта

Заголовочный файл `calc/Interpolation.hpp` — публичный API библиотеки:

```
#pragma once

#include <cstddef>
#include <vector>

namespace interp {

enum class Method {
    Lagrange,
    NewtonForward,
    NewtonBackward,
    GaussI,
    GaussII,
    Stirling,
    Bessel
};

enum class Status { Ok, NotEquidistant, TooFew, Duplicate };

struct Point {
    double x = 0.0;
    double y = 0.0;
};

struct MethodResult {
    Method method = Method::Lagrange;
    Status status = Status::Ok;
    double value = 0.0;
    double t = 0.0;
    int center = 0;
    int order = 0;
};

constexpr std::size_t kMinPoints = 2;
constexpr std::size_t kMaxPoints = 30;

bool isEquidistant(const std::vector<Point> &nodes, double &h);

std::vector<std::vector<double>>
finiteDifferences(const std::vector<Point> &nodes);

double lagrange(const std::vector<Point> &nodes, double X);

MethodResult run(Method method, const std::vector<Point> &nodes, double X);

const char *methodKey(Method m);
const char *methodTitle(Method m);

} // namespace interp
```

Реализация (`calc/interpolation.cpp`) — все методы, таблица разностей, диспетчер `run`:

```

#include "Interpolation.hpp"

#include <cmath>

namespace interp {

namespace {

constexpr double kEps = 1e-9;

double factorial(int k) {
    double f = 1.0;
    for (int i = 2; i ≤ k; ++i) {
        f *= static_cast<double>(i);
    }
    return f;
}

int nearestIndex(const std::vector<Point> &nodes, double X) {
    int best = 0;
    double bestDist = std::abs(nodes[0].x - X);
    for (int i = 1; i < static_cast<int>(nodes.size()); ++i) {
        const double d = std::abs(nodes[i].x - X);
        if (d < bestDist) {
            bestDist = d;
            best = i;
        }
    }
    return best;
}

using Diff = std::vector<std::vector<double>>;

double newtonForward(const Diff &d, const std::vector<Point> &nodes, double h,
                    double X, int &order) {
    const int n = static_cast<int>(nodes.size());
    const double t = (X - nodes[0].x) / h;
    double result = nodes[0].y;
    double factor = 1.0;
    order = 0;
    for (int k = 1; k ≤ n - 1; ++k) {
        factor *= (t - (k - 1)) / k;
        result += factor * d[k][0];
        order = k;
    }
    return result;
}

double newtonBackward(const Diff &d, const std::vector<Point> &nodes, double h,
                     double X, int &order) {
    const int n = static_cast<int>(nodes.size());
    const double t = (X - nodes[n - 1].x) / h;
    double result = nodes[n - 1].y;
    double factor = 1.0;
    order = 0;
    for (int k = 1; k ≤ n - 1; ++k) {
        factor *= (t + (k - 1)) / k;
        result += factor * d[k][n - 1 - k];
        order = k;
    }
}

```

```

    return result;
}

double gaussForward(const Diff &d, const std::vector<Point> &nodes, double h,
                   double X, int c, double &t, int &order) {
    const int n = static_cast<int>(nodes.size());
    t = (X - nodes[c].x) / h;
    double result = nodes[c].y;
    double prod = 1.0;
    double fact = 1.0;
    order = 0;
    for (int k = 1;; ++k) {
        const int idx = c - k / 2;
        if (idx < 0 || idx + k > n - 1) {
            break;
        }
        const int half = k / 2;
        const double f = (k % 2 == 1) ? (t + half) : (t - half);
        prod *= f;
        fact *= k;
        result += prod / fact * d[k][idx];
        order = k;
    }
    return result;
}

double gaussBackward(const Diff &d, const std::vector<Point> &nodes, double h,
                    double X, int c, double &t, int &order) {
    const int n = static_cast<int>(nodes.size());
    t = (X - nodes[c].x) / h;
    double result = nodes[c].y;
    double prod = 1.0;
    double fact = 1.0;
    order = 0;
    for (int k = 1;; ++k) {
        const int idx = c - (k + 1) / 2;
        if (idx < 0 || idx + k > n - 1) {
            break;
        }
        const int half = k / 2;
        const double f = (k % 2 == 1) ? (t - half) : (t + half);
        prod *= f;
        fact *= k;
        result += prod / fact * d[k][idx];
        order = k;
    }
    return result;
}

double stirling(const Diff &d, const std::vector<Point> &nodes, double h,
                double X, int c, double &t, int &order) {
    const int n = static_cast<int>(nodes.size());
    t = (X - nodes[c].x) / h;
    const double t2 = t * t;
    double result = nodes[c].y;
    double p2 = 1.0;
    double fact = 1.0;
    order = 0;
    for (int k = 1;; ++k) {
        fact *= k;

```

```

    if (k % 2 == 1) {
        const int j = (k + 1) / 2;
        const int idxA = c - j;
        const int idxB = c - (j - 1);
        if (idxA < 0 || idxB + k > n - 1) {
            break;
        }
        const double diff = (d[k][idxA] + d[k][idxB]) / 2.0;
        result += t * p2 / fact * diff;
    } else {
        const int j = k / 2;
        const int idx = c - j;
        if (idx < 0 || idx + k > n - 1) {
            break;
        }
        result += t2 * p2 / fact * d[k][idx];
        p2 *= (t2 - static_cast<double>(j) * static_cast<double>(j));
    }
    order = k;
}
return result;
}

double bessel(const Diff &d, const std::vector<Point> &nodes, double h,
              double X, int c, double &t, int &order) {
    const int n = static_cast<int>(nodes.size());
    t = (X - nodes[c].x) / h;
    double result = (d[0][c] + d[0][c + 1]) / 2.0;
    order = 0;
    if (c + 1 ≤ n - 1) {
        result += (t - 0.5) * d[1][c];
        order = 1;
    }
    double g = 1.0;
    for (int j = 1; ; ++j) {
        g *= (t + (j - 1)) * (t - static_cast<double>(j));
        const int e = 2 * j;
        const int eA = c - j;
        const int eB = c - (j - 1);
        if (eA < 0 || eB + e > n - 1) {
            break;
        }
        const double diffEven = (d[e][eA] + d[e][eB]) / 2.0;
        result += g / factorial(e) * diffEven;
        order = e;
        const int o = 2 * j + 1;
        const int oA = c - j;
        if (oA < 0 || oA + o > n - 1) {
            break;
        }
        result += (t - 0.5) * g / factorial(o) * d[o][oA];
        order = o;
    }
    return result;
}

} // namespace

bool isEquidistant(const std::vector<Point> &nodes, double &h) {
    const std::size_t n = nodes.size();

```

```

    if (n < 2) {
        h = 0.0;
        return false;
    }
    h = nodes[1].x - nodes[0].x;
    if (std::abs(h) < kEps) {
        return false;
    }
    for (std::size_t i = 1; i < n; ++i) {
        const double step = nodes[i].x - nodes[i - 1].x;
        if (std::abs(step - h) > 1e-6 * std::max(1.0, std::abs(h))) {
            return false;
        }
    }
    return true;
}

std::vector<std::vector<double>>
finiteDifferences(const std::vector<Point> &nodes) {
    const std::size_t n = nodes.size();
    std::vector<std::vector<double>> d(n);
    d[0].resize(n);
    for (std::size_t i = 0; i < n; ++i) {
        d[0][i] = nodes[i].y;
    }
    for (std::size_t k = 1; k < n; ++k) {
        d[k].resize(n - k);
        for (std::size_t i = 0; i < n - k; ++i) {
            d[k][i] = d[k - 1][i + 1] - d[k - 1][i];
        }
    }
    return d;
}

double lagrange(const std::vector<Point> &nodes, double X) {
    const std::size_t n = nodes.size();
    double sum = 0.0;
    for (std::size_t i = 0; i < n; ++i) {
        double term = nodes[i].y;
        for (std::size_t j = 0; j < n; ++j) {
            if (j == i) {
                continue;
            }
            term *= (X - nodes[j].x) / (nodes[i].x - nodes[j].x);
        }
        sum += term;
    }
    return sum;
}

MethodResult run(Method method, const std::vector<Point> &nodes, double X) {
    MethodResult r;
    r.method = method;

    const int n = static_cast<int>(nodes.size());
    if (n < static_cast<int>(kMinPoints)) {
        r.status = Status::TooFew;
        return r;
    }
    for (int i = 0; i < n; ++i) {

```

```

    for (int j = i + 1; j < n; ++j) {
        if (std::abs(nodes[i].x - nodes[j].x) < kEps) {
            r.status = Status::Duplicate;
            return r;
        }
    }
}

if (method == Method::Lagrange) {
    r.value = lagrange(nodes, X);
    r.order = n - 1;
    return r;
}

double h = 0.0;
if (!isEquidistant(nodes, h)) {
    r.status = Status::NotEquidistant;
    return r;
}
const Diff d = finiteDifferences(nodes);

switch (method) {
case Method::NewtonForward:
    r.center = 0;
    r.t = (X - nodes[0].x) / h;
    r.value = newtonForward(d, nodes, h, X, r.order);
    break;
case Method::NewtonBackward:
    r.center = n - 1;
    r.t = (X - nodes[n - 1].x) / h;
    r.value = newtonBackward(d, nodes, h, X, r.order);
    break;
case Method::GaussI:
    r.center = nearestIndex(nodes, X);
    r.value = gaussForward(d, nodes, h, X, r.center, r.t, r.order);
    break;
case Method::GaussII:
    r.center = nearestIndex(nodes, X);
    r.value = gaussBackward(d, nodes, h, X, r.center, r.t, r.order);
    break;
case Method::Stirling:
    r.center = nearestIndex(nodes, X);
    r.value = stirling(d, nodes, h, X, r.center, r.t, r.order);
    break;
case Method::Bessel: {
    int c = static_cast<int>(std::floor((X - nodes[0].x) / h));
    if (c < 0) {
        c = 0;
    }
    if (c > n - 2) {
        c = n - 2;
    }
    r.center = c;
    r.value = besse1(d, nodes, h, X, c, r.t, r.order);
    break;
}
case Method::Lagrange:
    break;
}

```

```

    return r;
}

const char *methodKey(Method m) {
    switch (m) {
        case Method::Lagrange:
            return "lagrange";
        case Method::NewtonForward:
            return "newton_fwd";
        case Method::NewtonBackward:
            return "newton_bwd";
        case Method::GaussI:
            return "gauss1";
        case Method::GaussII:
            return "gauss2";
        case Method::Stirling:
            return "stirling";
        case Method::Bessel:
            return "bessel";
    }
    return "";
}

const char *methodTitle(Method m) {
    switch (m) {
        case Method::Lagrange:
            return "Многочлен Лагранжа";
        case Method::NewtonForward:
            return "Ньютон (вперёд)";
        case Method::NewtonBackward:
            return "Ньютон (назад)";
        case Method::GaussI:
            return "Гаусс (1-я формула)";
        case Method::GaussII:
            return "Гаусс (2-я формула)";
        case Method::Stirling:
            return "Стирлинг";
        case Method::Bessel:
            return "Бессель";
    }
    return "";
}

} // namespace interp

```


5.5 Интерфейс приложения



Рис. 11. Внешний вид вкладки «Интерполяция» в приложении
Полный исходный код проекта (Qt6/QML/C++ десктоп-приложение) доступен в репозитории:

`git.oriptal.dev/cadmin/calc-math-lab2`

6 Выводы

В ходе лабораторной работы изучены и программно реализованы методы интерполяции табличной функции: многочлен Лагранжа, интерполяционные формулы Ньютона (первая и вторая, с конечными разностями), формулы Гаусса (первая и вторая), а также дополнительно схемы Стирлинга и Бесселя. Для всех методов программа выводит значение в заданной точке, параметр t , фактическую степень и пояснение о применимости; таблица конечных разностей строится автоматически, а график отображает узлы, интерполяционный многочлен и заданную функцию разными цветами.

Для варианта №5 (таблица 1.5, семь узлов, $h = 0.05$) в вычислительной части получены значения: в точке $X_1 = 2.112$ по первой

формуле Ньютона и по Лагранжу — 3.645469 (совпадение полное, так как строится один и тот же многочлен 6-й степени); в точке $X_2 = 2.205$ по первой формуле Гаусса — 4.973381, по Лагранжу — 4.968647 (расхождение ≈ 0.005 из-за разной степени усечения центральной формулы). Совпадение методов на полном наборе узлов подтверждает теорему о единственности интерполяционного многочлена.

Отмечено, что конечные разности заданной таблицы не убывают, поэтому многочлен 6-й степени носит колебательный характер (в точке X_1 , расположенной сразу за узлом $x_0 = 2.10$, его значение 3.645469 оказывается даже ниже $y_0 = 3.7587$). Это типичное проявление неустойчивости глобальной интерполяции многочленом высокой степени при нерегулярных данных; на практике в таких случаях предпочтительнее локальная интерполяция по нескольким ближайшим узлам или центральные схемы (Гаусса, Стирлинга), дающие меньшую степень и лучшую обусловленность вблизи центра таблицы. Корректность реализации проверена сверкой значений всех методов между собой и с независимым расчётом многочлена 6-й степени.